

LAPORAN TEKNIS DAN PANDUAN PENGGUNA

Proyek Berbasis Masalah, Mata Kuliah Keamanan Informasi (KOM1315)

Sistem Keamanan Aplikasi TUMBUH

Penerapan Protokol AAA, Enkripsi Data, Tanda Tangan Digital,
dan Audit Trail pada Platform Karier dan Magang IPB



Kelompok 9

Ghiffari Bravia Hisham (NIM: G640123050)

Adam Naufal (NIM: G6401231083)

Rafif Muhammad Farras (NIM: G6401231102)

PROGRAM STUDI ILMU KOMPUTER
INSTITUT PERTANIAN BOGOR
2026

Daftar Isi

1	Pendahuluan	2
1.1	Tujuan Dokumen	2
1.2	Ikhtisar Sistem	2
1.3	Ringkasan Fitur Keamanan	2
2	Arsitektur Sistem	3
2.1	Komponen	3
2.2	Tumpukan Teknologi	4
3	Implementasi Fitur Keamanan	4
3.1	Authentication	4
3.1.1	Registrasi dan Login	4
3.1.2	JSON Web Token	4
3.2	Authorization	5
3.3	Accounting (Audit Trail)	5
3.4	Password Hashing dan Salting	6
3.5	Enkripsi Tingkat Kolom	6
3.6	Tanda Tangan Digital	6
3.7	Kontrol Pendukung	7
4	Manajemen Kunci dan Rahasia	7
5	Pengujian Keamanan	8
6	Panduan Instalasi	9
6.1	Prasyarat	9
6.2	Konfigurasi Lingkungan Backend	9
6.3	Basis Data Lokal	9
6.4	Menjalankan Backend	9
6.5	Menjalankan Frontend	10
6.6	Menjalankan Layanan Audit	10
7	Panduan Penggunaan	10
7.1	Mahasiswa	10
7.2	HR Perusahaan	10
7.3	Admin	11
8	Penutup	11
A	Lampiran: Struktur Repositori	11
B	Lampiran: Keluaran Pengujian	11

1 Pendahuluan

1.1 Tujuan Dokumen

Laporan ini menjelaskan bagaimana keamanan diterapkan pada aplikasi TUMBUH, sekaligus menjadi panduan untuk menjalankan dan memakainya. Pembahasan dimulai dari arsitektur sistem, lalu cara setiap kontrol keamanan bekerja, hasil pengujian, dan diakhiri dengan langkah instalasi serta pemakaian. Pembaca yang kami tuju adalah pengembang, dosen penilai, dan pengguna aplikasi, baik mahasiswa, HR, maupun admin.

1.2 Ikhtisar Sistem

TUMBUH adalah aplikasi web pelacak karier dan magang untuk sivitas IPB. Aplikasi ini melayani tiga peran pengguna yang kebutuhan aksesnya berbeda, seperti dirangkum pada Tabel 1. Data yang ditangani tergolong sensitif, mulai dari profil pribadi, dokumen CV, surat lamaran, sampai status seleksi. Itulah alasan keamanannya dibangun di atas kerangka AAA (*Authentication, Authorization, Accounting*), lalu dilengkapi *password hashing*, enkripsi data, dan tanda tangan digital.

Tabel 1: Peran pengguna dan kebutuhan akses

Peran	Kebutuhan akses utama
Mahasiswa (student)	Membuat profil dan CV, mencari serta menyimpan lowongan, mengirim lamaran, memantau status.
HR (hr)	Mengelola profil perusahaan, membuat lowongan, meninjau pelamar, memperbarui status seleksi.
Admin (admin)	Mengawasi pengguna, perusahaan, dan lowongan; memverifikasi tanda tangan digital serta integritas audit.

1.3 Ringkasan Fitur Keamanan

Tabel 2 mengikhtisarkan kontrol keamanan yang sudah berjalan beserta teknologi pendukungnya. Penjelasan lebih rinci untuk masing-masing kontrol dapat ditemukan pada Bagian 3.

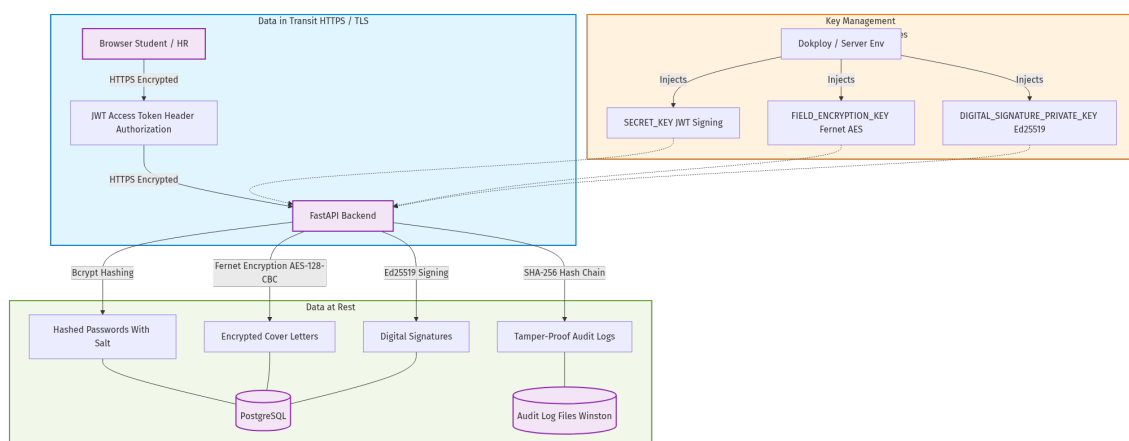
Tabel 2: Ringkasan fitur keamanan dan teknologinya

Aspek	Teknologi	Status
Password hashing	bcrypt (auto-salt, 12 putaran)	Aktif
Autentikasi token	JWT HS256 (access dan refresh)	Aktif
Login pihak ketiga	Google OAuth (verifikasi ID token)	Aktif
Otorisasi	RBAC, ownership check, kontrol CV	Aktif
Enkripsi data	Fernet (AES-128-CBC dan HMAC-SHA256)	Aktif
Tanda tangan digital	Ed25519 pada peristiwa lamaran	Aktif
Audit trail	Winston dengan rantai hash SHA-256	Aktif
Kontrol pendukung	Security headers, rate limit, CORS, upload	Aktif
Manajemen rahasia	Variabel lingkungan, tanpa hardcoded key	Aktif

2 Arsitektur Sistem

2.1 Komponen

TUMBUH dibangun dari empat komponen yang saling terhubung. Frontend (**fe-web/**) memakai React dan Vite; di sinilah JWT disimpan dan dikirim lewat tajuk **Authorization: Bearer**. Backend (**be-web/**) berbasis FastAPI dengan SQLAlchemy dan Alembic, menjadi tempat seluruh logika bisnis dan keamanan berjalan, termasuk enkripsi dan penandatanganan. Layanan audit (**audit-log/**) berbasis Express dan Winston menerima peristiwa secara *fire-and-forget* lalu merantainya dengan hash. Adapun basis datanya memakai PostgreSQL, baik melalui Docker lokal maupun Supabase. Hubungan antarkomponen tersebut digambarkan pada Gambar 1.



Gambar 1: Arsitektur keamanan TUMBUH yang mencakup data at rest, in transit, dan manajemen kunci

2.2 Tumpukan Teknologi

Pemilihan pustaka pada Tabel 3 mengutamakan yang sudah matang dan menyediakan primitif kriptografi teruji, sehingga tim tidak perlu menulis ulang fungsi keamanan dari nol.

Tabel 3: Tumpukan teknologi dan pustaka keamanan

Lapisan	Teknologi	Peran keamanan
Frontend	React dan Vite	Penyimpanan token, request terautentikasi
Backend	FastAPI 0.115	Validasi Pydantic, dependency injection, AAA
ORM	SQLAlchemy 2.0	Parameterized query (mencegah SQL injection)
Hashing	bcrypt 4.2.0	Password hashing dengan salt
Token	python-jose 3.3.0	JWT HS256
Kripto	cryptography	Enkripsi kolom (Fernet) dan tanda tangan (Ed25519)
OAuth	google-auth 2.45.0	Verifikasi ID token Google
Rate limit	slowapi 0.1.9	Pembatasan laju per IP
Audit	Express 5, Winston 3	Logging dengan rantai hash SHA-256

3 Implementasi Fitur Keamanan

3.1 Authentication

Pengguna dapat masuk lewat tiga jalur: pasangan surel dan kata sandi, Google OAuth, atau sesi yang sudah berjalan melalui JWT.

3.1.1 Registrasi dan Login

Saat mendaftar, sistem memeriksa format surel, memastikan mahasiswa memakai domain institusi @ipb.ac.id, menolak surel yang sudah terpakai, lalu menyimpan kata sandi dalam bentuk hash dan mengirim tautan verifikasi. Pada proses login, kata sandi dicocokkan, akun dipastikan sudah terverifikasi dan masih aktif, barulah sistem menerbitkan *access token* berumur 60 menit dan *refresh token* berumur tujuh hari.

3.1.2 JSON Web Token

Token ditandatangani dengan algoritma HS256. Setiap permintaan ke sumber daya yang dilindungi akan membaca token tersebut untuk mengenali penggunanya, dan *refresh token* sengaja tidak bisa dipakai untuk memanggil API biasa. Tabel 4 merangkum endpoint autentikasi utama beserta batas lajunya.

Tabel 4: Endpoint autentikasi utama

Endpoint	Fungsi	Rate limit
POST /auth/register	Registrasi pengguna	5/menit
POST /auth/login	Login dan terbitkan token	5/menit
POST /auth/google	Login atau daftar via Google	10/menit
POST /auth/verify-email	Verifikasi surel	10/menit
POST /auth/refresh	Perbarui access token	tidak dibatasi

3.2 Authorization

Otorisasi memakai RBAC atas tiga peran. Pembatasannya ditangani oleh dependensi `require_role()` yang akan menolak permintaan dengan HTTP 403 ketika peran pengguna tidak cocok, seperti pada Kode 1.

```
def require_role(required_role: str):
    def _check_role(current_user: User = Depends(get_current_user)) -> User:
        if current_user.role.value != required_role:
            raise HTTPException(status_code=403,
                                detail=f"Access denied. Required role: {required_role}")
        return current_user
    return _check_role
```

Kode 1: Dependensi RBAC

Peran saja belum cukup, sehingga sistem menambahkan pemeriksaan kepemilikan. HR hanya boleh menangani pelamar pada lowongan milik perusahaannya sendiri. Dokumen CV pun dijaga ketat: hanya pemiliknya, admin, atau HR yang relevan yang bisa membukanya. CV disimpan di direktori privat (`private_uploads/`) dan tidak pernah disajikan sebagai berkas statis, sehingga setiap aksesnya selalu melewati lapisan autentikasi dan otorisasi.

3.3 Accounting (Audit Trail)

Setiap peristiwa penting dikirim ke layanan audit secara *fire-and-forget* pada utas latar belakang. Cara ini membuat pencatatan tidak memperlambat respons API dan tetap aman apabila layanan audit sedang mati. Keutuhan catatan dijaga dengan rantai hash SHA-256: tiap peristiwa di-hash bersama hash peristiwa sebelumnya, sehingga begitu ada catatan yang dipalsukan atau dihapus, rantainya langsung putus dan ketahuan, seperti diperlihatkan Kode 2.

```
function chainAuditEvent(event) {
    const previousHash = loadLastHash(); // 'GENESIS' bila pertama kali
    const eventHash = crypto.createHash('sha256')
        .update(previousHash + ':' + canonicalJson(event)).digest('hex');
    return { ...event, previousHash, eventHash };
}
```

Kode 2: Perantain hash audit

Admin dapat memeriksa keutuhan rantai ini kapan saja lewat endpoint GET /admin/security/audit/verif yang menelusuri kecocokan setiap tautan hash di seluruh berkas log.

3.4 Password Hashing dan Salting

Kata sandi tidak pernah disimpan apa adanya. Sistem memakai bcrypt dengan salt acak yang dibuat otomatis dan faktor biaya 12. Karena field `hashed_password` tidak pernah dicantumkan pada skema respons, nilainya otomatis tidak ikut keluar di balasan API mana pun. Kode 3 menunjukkan fungsi hashing dan verifikasi.

```
def hash_password(password: str) -> str:
    return _bcrypt.hashpw(password.encode(), _bcrypt.gensalt()).decode()

def verify_password(plain: str, hashed: str) -> bool:
    return _bcrypt.checkpw(plain.encode(), hashed.encode())
```

Kode 3: Hashing kata sandi

3.5 Enkripsi Tingkat Kolom

Surat lamaran termasuk data yang paling sensitif, sehingga isinya dienkripsi dengan Fernet (AES-128-CBC dipadu HMAC-SHA256). Nilai hasil enkripsi diberi awalan `enc:v1:` untuk menandai bahwa data sudah terenkripsi, mencegah enkripsi ganda, dan mempermudah migrasi bila algoritmanya berganti. Kode 4 memperlihatkan pasangan fungsinya.

```
def encrypt_text(self, value):
    if value is None or value.startswith(ENCRYPTED_PREFIX): return value
    return ENCRYPTED_PREFIX + self._fernet.encrypt(value.encode()).decode()

def decrypt_text(self, value):
    if value is None or not value.startswith(ENCRYPTED_PREFIX): return value
    try:
        return self._fernet.decrypt(value.removeprefix(ENCRYPTED_PREFIX).encode()).decode()
    except InvalidToken:
        return "[encrypted value could not be decrypted]"
```

Kode 4: Enkripsi dan dekripsi field

Di basis data, surat lamaran tersimpan sebagai *ciphertext*, sedangkan pada respons API isinya dikembalikan menjadi teks biasa setelah didekripsi. Dengan begitu, seandainya basis data bocor, isi surat lamaran tetap tidak terbaca karena penyerang masih membutuhkan kuncinya.

3.6 Tanda Tangan Digital

Peristiwa pada siklus lamaran ditandatangani memakai Ed25519. Sebelum ditandatangani, *payload* disusun menjadi JSON kanonik dengan kunci terurut dan tanpa spasi berlebih supaya hasilnya konsisten. Payload, tanda tangan, dan nama algoritmanya disimpan di basis data, dan admin dapat memverifikasinya lewat GET /admin/security/applications/{id}/signature.

Kode 5 menunjukkan proses tanda tangan dan verifikasi.

```
def sign_payload(self, payload) -> str:
    return base64.urlsafe_b64encode(
        self._private_key.sign(self._canonical_json(payload).encode())) .decode()

def verify_signature(self, payload, signature) -> bool:
    try:
        self._public_key.verify(base64.urlsafe_b64decode(signature),
                                self._canonical_json(payload).encode())

        return True
    except Exception:
        return False
```

Kode 5: Tanda tangan dan verifikasi Ed25519

Karena payload memuat `actor_id` dan `timestamp`, mahasiswa tidak bisa menyangkal telah mengirim lamaran, begitu pula HR atas perubahan status yang ia lakukan. Jika payload diubah sedikit saja setelah ditandatangani, verifikasi langsung gagal.

3.7 Kontrol Pendukung

Beberapa kontrol tambahan melengkapi pertahanan sistem. Setiap respons membawa *security headers* (`X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, dan `Referrer-Policy`) untuk meredam *MIME sniffing* dan *clickjacking*. Endpoint autentikasi dibatasi lajunya per alamat IP, yakni 5 sampai 10 permintaan per menit, agar tebak paksa sulit dilakukan. CORS hanya membuka origin yang masuk daftar putih. Unggahan berkas diperiksa tipe dan ukurannya, dengan batas 2 MB untuk avatar dan 5 MB untuk CV berformat PDF, lalu namanya dibangkitkan ulang. Token verifikasi surel disimpan dalam bentuk hash SHA-256, bersifat sekali pakai, dan kedaluwarsa dalam 24 jam. Terakhir, seluruh masukan divalidasi oleh skema Pydantic di tiap endpoint.

4 Manajemen Kunci dan Rahasia

Semua kunci dan kredensial dipasok lewat variabel lingkungan, bukan ditulis langsung di kode. Berkas `.env` dikecualikan dari repositori melalui `.gitignore`. Tabel 5 mendaftar variabel rahasia yang dipakai sistem.

Tabel 5: Variabel lingkungan rahasia

Variabel	Fungsi	Tingkat
SECRET_KEY	Penandatanganan JWT dan derivasi kunci cadangan	Kritis
FIELD_ENCRYPTION_KEY	Kunci enkripsi Fernet (Base64 44 karakter)	Kritis
DIGITAL_SIGNATURE_PRIVATE_KEY	Benih kunci privat Ed25519	Kritis
POSTGRES_PASSWORD	Kata sandi basis data	Kritis
GOOGLE_CLIENT_ID	Verifikasi OAuth Google	Tinggi
AUDIT_DASHBOARD_KEY	Akses dashboard audit	Tinggi
CORS_ORIGINS	Daftar origin frontend	Sedang

Berkas `.env` maupun kunci kriptografi tidak boleh ikut *ter-commit* ke repositori. Untuk lingkungan produksi, bangkitkan `FIELD_ENCRYPTION_KEY` dan `DIGITAL_SIGNATURE_PRIVATE_KEY` yang kuat lalu masukkan lewat variabel lingkungan, jangan menurunkannya dari `SECRET_KEY` seperti pada mode pengembangan.

5 Pengujian Keamanan

Pengujian unit dijalankan dengan `pytest` memakai basis data SQLite *in-memory*, sehingga tidak bergantung pada infrastruktur produksi. Seluruh sembilan kasus uji lulus, dengan rincian pada Tabel 6.

Tabel 6: Hasil pengujian fungsi keamanan

Kasus uji	Aspek	Hasil
Enkripsi lalu dekripsi surat lamaran kembali utuh	Confidentiality	Lulus
Pemalsuan tanda tangan digital terdeteksi	Integrity	Lulus
Penonaktifan akun memblokir login berikutnya	Authentication	Lulus
Penghapusan akun memerlukan kecocokan surel	Authentication	Lulus
Penggantian kata sandi via reset surel	Authentication	Lulus
Akun student ditolak pada sumber daya admin (403)	Authorization	Lulus
Akun admin diizinkan pada sumber daya admin	Authorization	Lulus
Data terenkripsi tidak terbaca sebagai teks biasa	Confidentiality	Lulus
Ciphertext yang dirusak ditolak saat dekripsi	Integrity	Lulus

Pengujian dijalankan dengan perintah berikut.

```
cd be-web
export PYTHONPATH=. SECRET_KEY="test-secret-key"
```

```
python -m pytest tests/test_security_service.py tests/test_account_security.py -v
```

Kode 6: Menjalankan pengujian keamanan

Dua kasus terakhir, yaitu penolakan akses akun student dengan kode 403 dan penolakan *ciphertext* yang sudah dirusak, secara langsung menjawab ketentuan bahwa pengguna tanpa hak harus gagal mengakses data terenkripsi. Log lengkap pelaksanaannya tersedia pada folder `05_Testing_Logs/`.

6 Panduan Instalasi

Bagian ini memandu menjalankan TUMBUH pada lingkungan pengembangan lokal.

6.1 Prasyarat

Pastikan tersedia Node.js dan npm, Python 3.11 atau lebih baru, serta Docker Desktop untuk menjalankan PostgreSQL lokal. Akses terminal dan klien Git juga diperlukan.

6.2 Konfigurasi Lingkungan Backend

Salin berkas contoh lalu sesuaikan `DATABASE_URL`.

```
cd be-web
cp .env.example .env
# Untuk PostgreSQL Docker lokal:
# DATABASE_URL=postgres://postgres:postgres@localhost:5433/career_tracker
```

Kode 7: Menyiapkan .env backend

6.3 Basis Data Lokal

Jalankan PostgreSQL melalui Docker Compose dari akar repositori.

```
docker compose up -d postgres
```

Kode 8: Menjalankan PostgreSQL via Docker

Kontainer ini memakai basis data `career_tracker`, pengguna `postgres`, dan port 5433.

6.4 Menjalankan Backend

Pasang dependensi, terapkan migrasi, lalu jalankan server API.

```
cd be-web
pip install -r requirements.txt
alembic upgrade head # migrasi skema basis data
python -m scripts.seed # data awal (opsional)
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Kode 9: Memasang dan menjalankan API

Dokumentasi API interaktif (Swagger) terbuka di `http://127.0.0.1:8000/docs`.

6.5 Menjalankan Frontend

```
cd fe-web
npm install
npm run dev # http://localhost:5173
```

Kode 10: Menjalankan frontend

6.6 Menjalankan Layanan Audit

Layanan audit bersifat opsional; backend tetap berjalan walau layanan ini mati. Peristiwa audit akan terkirim begitu `AUDIT_LOG_URL` dikonfigurasi.

```
cd audit-log
npm install
npm start # http://localhost:3001
```

Kode 11: Menjalankan layanan audit

Untuk *deployment* penuh yang menyatukan frontend, backend, layanan audit, dan PostgreSQL sekaligus, gunakan `docker-compose.dokploy.yml` dengan `.env.dokploy.example` sebagai acuan variabel lingkungannya.

7 Panduan Penggunaan

Bagian ini menjelaskan alur pemakaian untuk tiap peran.

7.1 Mahasiswa

1. Daftar memakai surel institusi `@ipb.ac.id`, lalu klik tautan verifikasi yang dikirim ke surel.
2. Masuk untuk memulai sesi, lengkapi profil, dan unggah CV dalam format PDF (maksimal 5 MB).
3. Cari dan simpan lowongan, kemudian kirim lamaran. Surat lamaran otomatis dienkrpsi saat disimpan dan peristiwanya ditandatangani secara digital.
4. Pantau perkembangan status lamaran dari dasbor pribadi.

7.2 HR Perusahaan

1. Masuk ke akun HR. Saat registrasi, HR wajib mengisi nama resmi karena sistem tidak mengambilnya begitu saja dari nama Google.
2. Kelola profil perusahaan dan buat lowongan baru.
3. Tinjau pelamar, tetapi hanya untuk lowongan milik perusahaan sendiri. CV pelamar baru bisa dibuka apabila mahasiswa memang melamar ke perusahaan tersebut.
4. Perbarui status seleksi. Setiap perubahan ditandatangani secara digital dan tercatat di audit trail.

7.3 Admin

1. Masuk sebagai admin untuk membuka panel administrasi.
2. Kelola pengguna, perusahaan, dan lowongan, termasuk mengaktifkan, menonaktifkan, atau menghapusnya.
3. Verifikasi tanda tangan digital sebuah lamaran melalui menu keamanan admin.
4. Periksa keutuhan audit dengan verifikasi rantai hash dan pantau aliran peristiwa lewat dasbor audit.

Akses lintas peran ditolak secara otomatis dengan kode HTTP 403. Sebagai contoh, akun mahasiswa yang mencoba membuka endpoint admin akan langsung ditolak, dan dokumen CV hanya terbuka bagi pihak yang memang berwenang.

8 Penutup

Komponen keamanan TUMBUH sudah menerapkan kerangka AAA dan menyatukannya pada alur utama aplikasi, dengan tambahan *password hashing*, enkripsi tingkat kolom, tanda tangan digital, serta audit trail berantai hash. Seluruh fungsi keamanan inti lolos pengujian. Ke depan, sistem masih bisa diperkuat melalui rotasi kunci berkala, perluasan enkripsi ke atribut sensitif lain, autentikasi dua faktor, serta pengujian integrasi dan penetrasi.

A Lampiran: Struktur Repositori

```
KOM1315_SmtGenap26_Kelompok9_TUMBUH/
|-- 01_Proposal_&_Analisis/ # Proposal dan threat modeling
|-- 02_Design_Documents/ # ERD, arsitektur, rencana uji
|-- 03_Source_Code/
| |-- backend/ # Modul kriptografi (encrypt/decrypt)
| |-- database/ # Skema DB dan hashing/salting
| |-- digital_signature/ # Tanda tangan digital (Ed25519)
| '-- src/auth/ # Authentication dan RBAC
|-- 04_Reports_&_Paper/
| |-- Monitoring_P7/ # Laporan kemajuan
| |-- Final_Technical_Report/ # Dokumen ini dan User Manual
| '-- Scientific_Paper/ # Paper ilmiah (JIKA)
|-- 05_Testing_Logs/ # Log pengujian keamanan
'-- README.md
```

Kode 12: Struktur folder repositori

B Lampiran: Keluaran Pengujian

```
Suite 1: test_security_service.py + test_account_security.py
  test_encrypt_decrypt_text_roundtrip ..... PASSED
  test_digital_signature_detects_tampering ..... PASSED
  test_deactivate_account_blocks_future_login .... PASSED
```

```
test_delete_account_requires_matching_email .... PASSED
test_password_reset_email_allows_change ..... PASSED
=> 5 passed

Suite 2: test_authorization_and_encryption.py
[AUTHZ] admin -> admin resource : DIIZINKAN 200 .... PASSED
[CONF ] plaintext disembunyikan (enc:v1:...) ..... PASSED
[AUTHZ] student -> admin resource : DITOLAK 403 ..... PASSED
[INTEG] ciphertext dirusak -> dekripsi gagal ..... PASSED
=> 4 passed

TOTAL: 9 passed, 0 failed.
```

Kode 13: Ringkasan keluaran pytest